

Boletín de ejercicios — Planificación de procesos

Sistemas Operativos · Tema 2: Procesos · Algoritmos FCFS y Round Robin

Tema 2 — Procesos

FCFS · Round Robin

Dificultad creciente

7 ejercicios + teoría

Recordatorio de fórmulas (las usadas en clase):

- $T. Respuesta = Fin - Llegada$
- $T. Espera = T. Respuesta - T. Ejecución$
- $Penalización = T. Respuesta / T. Ejecución$

Convenios: «No expulsivo» = el proceso agota su quantum aunque llegue otro más prioritario.
«Expulsivo» = se expulsa si llega/existe uno de mayor prioridad. En empate de llegada entra antes el de mayor prioridad (y si hay empate, el que lleve más tiempo esperando).

Las soluciones detalladas paso a paso están al final del boletín. Dibuja tú mismo los cronogramas y rellena las tablas antes de mirarlas.

Parte A — Enunciados

Resuelve cada ejercicio dibujando el cronograma (diagrama de tiempos), la tabla de planificación (traza de colas) y la tabla de tiempos final.

Ejercicio 1 — FCFS (no expulsivo, sin prioridad)

Datos: Planificador FCFS (First Come First Served). El proceso se ejecuta de forma ininterrumpida hasta terminar. Todos llegan según la tabla:

Proceso	Llegada	T. ejecución
A	0	4
B	1	3
C	2	5
D	4	2

Se pide: (a) cronograma; (b) tabla de tiempos con T. respuesta, T. espera y penalización para cada proceso; (c) penalización media del sistema.

Ejercicio 2 — Round Robin (no expulsivo, sin prioridad, $q = 2$)

Datos: Round Robin con quantum = 2. Sin prioridades. Cuando un proceso agota su quantum vuelve al final de la cola de listos. En empate, entra antes el que llegó primero.

Proceso	Llegada	T. ejecución
A	0	5
B	1	3
C	2	4

D	3	2
---	---	---

Se pide: cronograma, tabla de planificación (traza de la cola en cada instante) y tabla de tiempos final.

Ejercicio 3 — Round Robin con prioridad (no expulsivo, q = 3)

Datos: Round Robin con quantum = 3 y prioridades estáticas. **No expulsivo:** aunque llegue un proceso más prioritario, el que está en CPU agota su quantum; al reorganizar la cola, se atiende según prioridad (1 = más prioritaria).

Proceso	Llegada	T. ejecución	Prioridad
A	0	4	2
B	1	5	1
C	3	2	3
D	4	3	1

Se pide: cronograma, tabla de planificación y tabla de tiempos. Indica claramente cómo se ordena la cola cada vez que se libera la CPU.

Ejercicio 4 — Round Robin con prioridad (EXPULSIVO, q = 2)

Datos: Round Robin con quantum = 2 y prioridades estáticas. **Expulsivo:** si llega un proceso de prioridad mayor que el que está ejecutándose, lo expulsa inmediatamente (1 = más prioritaria).

Proceso	Llegada	T. ejecución	Prioridad
P	0	7	3
Q	3	4	1
R	5	3	2
S	6	6	2

Se pide: cronograma, tabla de planificación y tabla de tiempos. Señala los instantes en que se produce una expulsión por prioridad.

Ejercicio 5 — Comparativa FCFS vs Round Robin

Datos: Con el mismo conjunto de procesos, resuelve dos veces: primero con **FCFS** y luego con **Round Robin** $q = 3$ (no expulsivo, sin prioridad).

Proceso	Llegada	T. ejecución
A	0	6
B	2	4
C	3	2
D	5	5

Se pide: tabla de tiempos completa para cada algoritmo y compara la **penalización media**. ¿Cuál ofrece mejor tiempo de respuesta medio? Razónalo.

Ejercicio 6 — Estilo examen (Round Robin, no expulsivo, con prioridad, $q = 4$)

Datos: Round Robin con quantum = 4, prioridad estática, no expulsivo (agota quantum y luego mira prioridad en la cola). 1 = más prioritaria.

Proceso	Llegada	T. ejecución	Prioridad
L	0	5	2
M	2	8	1
N	4	3	1
O	6	6	3

Se pide: (1) justifica el cronograma; (2) completa la tabla de planificación; (3) completa la tabla de tiempos. Usa estas plantillas en blanco:

Tabla de planificación (rellena tú):

T (CPU)	Llega	Cola de listos	Finaliza

Tabla de tiempos (rellena tú):

Proc.	Llega	T. ejec	Inicio	Fin	T. Resp	T. Esp	Penaliz.

Ejercicio 7 — Preguntas teóricas (Tema 2)

7.1. Define con tus palabras la diferencia entre planificación **expulsiva** y **no expulsiva**. Pon un ejemplo de algoritmo de cada tipo.

7.2. ¿Qué es la **inanición (starvation)** y en qué situación de Round Robin con prioridades puede aparecer? Propón una forma de mitigarla.

7.3. Explica qué es el **proceso nulo** y cuándo se ejecuta.

7.4. El planificador **CFS** de Linux usa un árbol binario (rojo-negro) ordenado por *vruntime*. ¿Qué proceso elige ejecutar en cada momento y por qué eso garantiza «equidad»?

7.5. En el ciclo de estados, ¿por qué un proceso que se desbloquea (acaba su E/S) pasa a **Listo** y no directamente a **Ejecutando**? Relaciónalo con el papel del *scheduler* y el *dispatcher*.

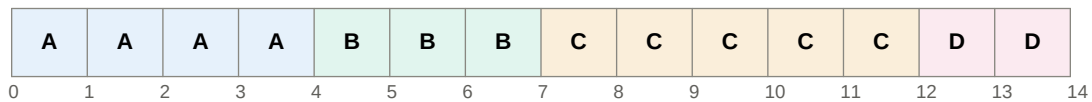
Parte B — Soluciones detalladas

Resolución paso a paso. Las trazas siguen el formato visto en clase: cronograma, tabla de planificación (cola en cada instante) y tabla de tiempos con las fórmulas $T.Resp = Fin - Lleg$, $T.Esp = T.Resp - T.ejec$, $Penaliz = T.Resp / T.ejec$.

Solución 1 — FCFS

Orden de ejecución por llegada: $A \rightarrow B \rightarrow C \rightarrow D$ (cada proceso corre entero).

Cronograma:



Proc.	Llega	T.ejec	Inicio	Fin	T.Resp	T.Esp	Penaliz.
A	0	4	0	4	4	0	1,00
B	1	3	4	7	6	3	2,00
C	2	5	7	12	10	5	2,00
D	4	2	12	14	10	8	5,00

Penalización media = $(1,00 + 2,00 + 2,00 + 5,00) / 4 = 2,50$. Observa cómo D, siendo el más corto, sufre la mayor penalización: es el efecto convoy típico de FCFS.

Solución 2 — Round Robin $q = 2$ (sin prioridad)

Regla: el proceso corre como máximo 2 u.; al expirar el quantum vuelve al final de la cola, pero **después** de los que han llegado durante ese quantum.

Cronograma:

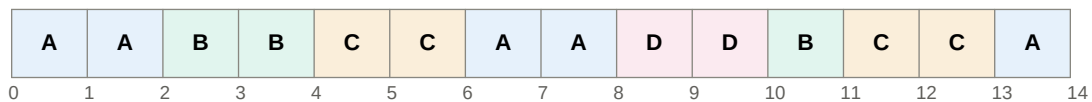


Tabla de planificación (cola de listos en cada cambio de CPU):

T (CPU)	CPU	Llega	Cola de listos tras reorganizar	Finaliza
0	A	A	[A]	—
2	B	B, C	[B, C, A]	—
4	C	D	[C, A, D, B]	—
6	A	—	[A, D, B, C]	—
8	D	—	[D, B, C, A]	—
10	B	—	[B, C, A]	D (t=10)
11	C	—	[C, A]	B (t=11)
13	A	—	[A]	C (t=13)
14	—	—	[]	A (t=14)

Tabla de tiempos:

Proc.	Llega	T.ejec	Fin	T.Resp	T.Esp	Penaliz.
A	0	5	14	14	9	2,80

B	1	3	11	10	7	3,33
C	2	4	13	11	7	2,75
D	3	2	10	7	5	3,50

Penalización media = $(2,80 + 3,33 + 2,75 + 3,50) / 4 \approx \mathbf{3,10}$.

Solución 3 — Round Robin q = 3 con prioridad (no expulsivo)

1 = más prioritaria. El proceso en CPU agota su quantum (no se le expulsa); cuando la CPU queda libre se elige de la cola el de **menor número de prioridad**; en empate, el que llegó antes.

Cronograma:

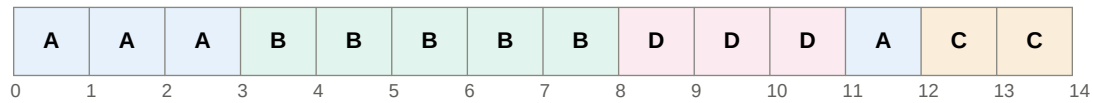


Tabla de planificación (cola ya ordenada por prioridad; en empate, por llegada):

T (CPU)	CPU	Llega	Cola de listos (orden prioridad)	Finaliza
0	A	A	[A]	—
3	B	B, C	[B(1), A(2), C(3)]	—
6	B	D	[B(1), D(1), A(2), C(3)]	—
8	D	—	[D(1), A(2), C(3)]	B (t=8)
11	A	—	[A(2), C(3)]	D (t=11)
12	C	—	[C(3)]	A (t=12)
14	—	—	[]	C (t=14)

Tabla de tiempos:

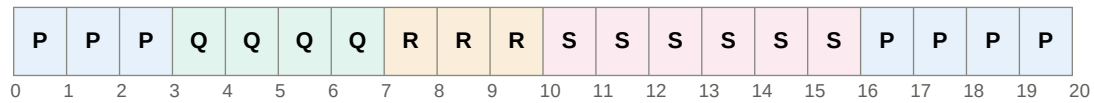
Proc.	Llega	T.ejec	Prior.	Fin	T.Resp	T.Esp	Penaliz.
A	0	4	2	12	12	8	3,00
B	1	5	1	8	7	2	1,40
C	3	2	3	14	11	9	5,50
D	4	3	1	11	7	4	2,33

Observa cómo C, con la peor prioridad (3), es el más penalizado (5,50): es el candidato a sufrir inanición si llegaran continuamente procesos prioritarios.

Solución 4 — Round Robin q = 2 con prioridad (EXPULSIVO)

1 = más prioritaria. Si llega un proceso de prioridad mayor que el que corre, lo expulsa de inmediato (sin esperar al quantum). Dentro de igual prioridad, el quantum y el turno funcionan como RR normal.

Cronograma (marca ↑ expulsión de P en t=3):



En t = 3 llega Q (prioridad 1) y expulsa a P (prioridad 3), que solo había ejecutado 3 de sus 7 unidades. P no vuelve a CPU hasta que terminan todos los procesos más prioritarios.

Tabla de planificación:

T (CPU)	CPU	Llega	Cola de listos (orden prioridad)	Finaliza
0	P	P	[P(3)]	—
3	Q	Q	[Q(1)] · P expulsado	—
5	Q	R	[Q(1), R(2), P(3)]	—
7	R	S	[R(2), S(2), P(3)]	Q (t=7)
10	S	—	[S(2), P(3)]	R (t=10)
16	P	—	[P(3)]	S (t=16)
20	—	—	[]	P (t=20)

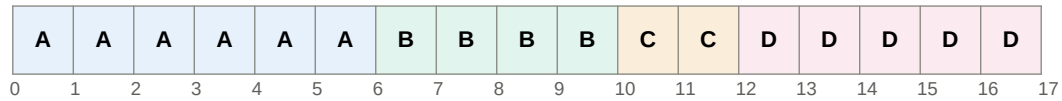
Tabla de tiempos:

Proc.	Llega	T.ejec	Prior.	Fin	T.Resp	T.Esp	Penaliz.
P	0	7	3	20	20	13	2,86
Q	3	4	1	7	4	0	1,00
R	5	3	2	10	5	2	1,67
S	6	6	2	16	10	4	1,67

Solución 5 — Comparativa FCFS vs RR (q = 3)

(a) **FCFS** — orden por llegada A → B → C → D:

Cronograma FCFS:

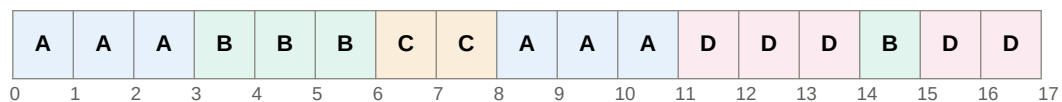


Proc.	Llega	T.ejec	Fin	T.Resp	T.Esp	Penaliz.
A	0	6	6	6	0	1,00
B	2	4	10	8	4	2,00
C	3	2	12	9	7	4,50
D	5	5	17	12	7	2,40

Penalización media FCFS = $(1,00+2,00+4,50+2,40)/4 = 2,48$.

(b) **Round Robin q = 3** (no expulsivo, sin prioridad):

Cronograma RR:



Proc.	Llega	T.ejec	Fin	T.Resp	T.Esp	Penaliz.
A	0	6	11	11	5	1,83
B	2	4	15	13	9	3,25
C	3	2	8	5	3	2,50
D	5	5	17	12	7	2,40

Penalización media RR = $(1,83+3,25+2,50+2,40)/4 = 2,50$.

Conclusión: en este caso las penalizaciones medias son casi idénticas (2,48 vs 2,50), pero RR reparte mejor el tiempo de respuesta de los procesos cortos (C pasa de penalización 4,50 en FCFS a 2,50 en RR), a costa de penalizar algo más a B. FCFS favorece a los que llegan primero; RR mejora la equidad y la interactividad.

Solución 6 — Estilo examen (RR q = 4, prioridad, no expulsivo)

1 = más prioritaria. Agota quantum y al liberar CPU se elige por prioridad (empate → llegada).

Cronograma resuelto:

L	L	L	L	M	M	M	M	M	M	M	M	N	N	N	L	O	O	O	O	O	O	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22

Tabla de planificación:

T (CPU)	CPU	Llega	Cola de listos (orden prioridad)	Finaliza
0	L	L	[L(2)]	—
4	M	M, N	[M(1), N(1), L(2)]	—
8	M	O	[M(1), N(1), L(2), O(3)]	—
12	N	—	[N(1), L(2), O(3)]	M (t=12)
15	L	—	[L(2), O(3)]	N (t=15)
16	O	—	[O(3)]	L (t=16)
22	—	—	[]	O (t=22)

Tabla de tiempos:

Proc.	Llega	T.ejec	Prior.	Fin	T.Resp	T.Esp	Penaliz.
L	0	5	2	16	16	11	3,20
M	2	8	1	12	10	2	1,25
N	4	3	1	15	11	8	3,67
O	6	6	3	22	16	10	2,67

Penalización media = $(3,20+1,25+3,67+2,67)/4 \approx 2,70$.

Solución 7 — Teoría

7.1 Expulsiva vs no expulsiva. En la planificación **no expulsiva** un proceso, una vez en CPU, no es desalojado hasta que termina o agota su quantum, aunque aparezca otro más prioritario (ej.: FCFS, o RR no expulsivo). En la **expulsiva** el planificador puede retirar el proceso de la CPU antes de tiempo si llega otro de mayor prioridad o por política de tiempo (ej.: RR expulsivo por prioridad, SRTF).

7.2 Inanición (starvation). Es la situación en que un proceso no llega nunca (o tarda muchísimo) a ejecutarse porque siempre hay otros más prioritarios ocupando la CPU. En RR con prioridades, un proceso de prioridad baja puede quedar postergado indefinidamente si entran continuamente procesos de prioridad alta. Se mitiga con **envejecimiento (aging)**: ir aumentando la prioridad de un proceso cuanto más tiempo lleve esperando.

7.3 Proceso nulo. Es un proceso especial (idle) que el SO ejecuta cuando no hay ningún otro proceso listo. Mantiene la CPU ocupada de forma controlada (típicamente en bajo consumo) y simplifica el diseño del planificador, que siempre tiene «algo» que ejecutar.

7.4 CFS y vruntime. El Completely Fair Scheduler guarda los procesos en un árbol rojo-negro ordenado por *vruntime* (tiempo virtual de ejecución acumulado). En cada decisión elige el proceso con **menor vruntime**, es decir, el que menos CPU ha disfrutado hasta ahora. Como al ejecutarse su

vruntime crece y baja en el orden, ningún proceso acapara la CPU: eso es lo que produce la equidad.

7.5 Desbloqueo → Listo, no Ejecutando. Cuando termina la E/S, el evento despierta al proceso y lo coloca en la **cola de listos**. Decidir quién usa la CPU es competencia del *scheduler* (elige) y del *dispatcher* (hace el cambio de contexto). El proceso no puede «colarse» directamente a Ejecutando aunque la CPU esté libre: siempre debe pasar por Listo y esperar a que el planificador le conceda el turno. (En sistemas expulsivos por prioridad, si el recién desbloqueado es más prioritario que el actual, puede provocar la expulsión del que corre, pero el camino sigue siendo Bloqueado → Listo → Ejecutando.)